```
    {
        int x, y, v; cin >> x >> y >> v;
        x++, y++; // 下标从 1 开始计数
        a[x][y] += v; // 同一个位置有可能有多个目标
    }

    n = 5001;
    // 预处理前缀和矩阵
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + a[i][j];
        }
    }

    int ret = 0;
    m = min(m, n); // 如果 m 很大，相当于就是把整个区域全部摧毁
    // 枚举所有边长为 m 的正方形
    for(int x2 = m; x2 <= n; x2++)
    {
        for(int y2 = m; y2 <= n; y2++)
        {
            int x1 = x2 - m + 1, y1 = y2 - m + 1;
            ret = max(ret, f[x2][y2] - f[x1 - 1][y2] - f[x2][y1 - 1] + f[x1 - 1][y1 - 1]);
        }
    }

    cout << ret << endl;

    return 0;
}
```

5. 差分

前缀和与差分的核心思想是**预处理**，可以在暴力枚举的过程中，快速给出查询的结果，从而优化时间复杂度。

是经典的**用空间替换时间**的做法。

学完差分之后，大家会发现，**前缀和与差分是一对互逆的运算**。

5.1 一维差分

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】差分
>
> **难度系数：** ★

【题目描述】

> 给你一个长度为 n 的正数数组 $a_1, a_2, ..., a_n$
>
> 接下来对这个数组进行 m 次操作，每个操作包含三个参数 l, r, k，代表将数组 $a_l, a_{l+1}, ..., a_r$ 都加上 k。
>
> 请输出操作后的数组。

【输入描述】

> 第一行包含两个整数 n 和 m。
>
> 第二行包含 n 个整数表示 $a_1, a_2, ..., a_n$
>
> 接下来是 m 行，每行三个整数，分别代表每次操作的参数 l, r, k
>
> - $1 \le n, m \le 10^5$
> - $-10^9 \le a[i] \le 10^9$
> - $1 \le l \le r \le n$
> - $-10^9 \le k \le 10^9$

【输出描述】

> 输出 1 行，表示 m 次操作后的 $a_1, a_2, ..., a_n$

【示例一】

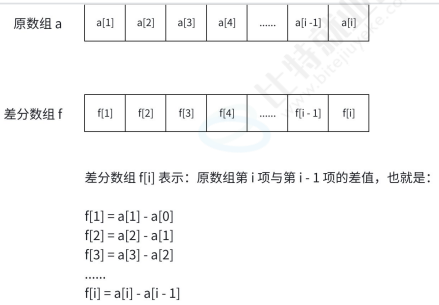
> 输入：
>
> 3 2
>
> 1 2 3
>
> 1 2 4
>
> 3 3 -2
>
> 输出：
>
> 5 6 1

【解法】

差分模板题，先「创建」差分数组，然后根据差分数组的「性质」处理 q 次区间修改，最后「还原」出来原始的数组。

1. 创建差分数组，根据定义： $f[i] = a[i] - a[i-1]$

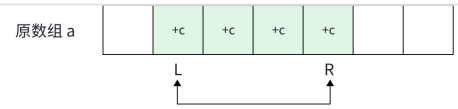
   也可以根据差分数组的性质： $f[i] \mathrel{+}= a[i],\ f[i+1] \mathrel{-}= a[i]$



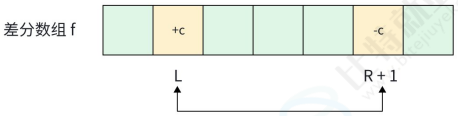
差分数组 f[i] 表示：原数组第 i 项与第 i - 1 项的差值，也就是：

f[1] = a[1] - a[0]
f[2] = a[2] - a[1]
f[3] = a[3] - a[2]
.....
f[i] = a[i] - a[i - 1]

2. 根据差分数组的性质处理 q 区间修改： $f[L] \mathrel{+}= c,\ f[R+1] \mathrel{-}= c$



每个元素都加上一个数 c



根据差分数组的定义，原数组 [L, R] 区间统一加上一个数 c 之后：

1. L 位置左边的每一个元素与前一个元素的差值不变，差分数组不变；
2. L 位置的元素与前一个元素的差值增加了 c，因此差分数组 f[L] += c；
3. L 位置与 R 位置之间的每一个元素与前一个元素的差值不变，差分数组不变；
4. R 位置右边的 R+1 元素与前一个元素的差值减少了 c，因此差分数组 f[R + 1] -= c；
5. R + 1 位置之后的差分值不变

3. 还原经过 q 次询问之后的 a 数组：对差分数组做一次「前缀和」，就可以还原出原数组

由差分数组的定义得：	原数组 a 中的每一项：
$f[1] = a[1]$	$a[1] = f[1]$
$f[2] = a[2] - a[1]$	$a[2] = a[2] - a[1] + a[1] = f[2] + f[1]$
$f[3] = a[3] - a[2]$	$a[3] = a[3] - a[2] + a[2] - a[1] + a[1] = f[3] + f[2] + f[1]$
.....	.....
$f[i] = a[i] - a[i-1]$	$a[i] = f[i] + f[i-1] + f[i-2] + ... + f[1]$

【参考代码】

代码块

```
#include <iostream>

using namespace std;

```

```cpp
typedef long long LL;

const int N = 1e5 + 10;

int n, m;
LL f[N]; // 差分数组

int main()
{
    cin >> n >> m;
    // 利用差分数组的性质，创建差分数组
    for(int i = 1; i <= n; i++)
    {
        LL x; cin >> x;
        f[i] += x;
        f[i + 1] -= x;
    }

    // 处理 m 次修改操作
    while(m--)
    {
        LL l, r, k; cin >> l >> r >> k;
        f[l] += k; f[r + 1] -= k;
    }

    // 还原出原始的数组
    for(int i = 1; i <= n; i++)
    {
        f[i] = f[i - 1] + f[i];
        cout << f[i] << " ";
    }

    return 0;
}
```

5.2 海底高铁

> **题目来源：** 洛谷
>
> **题目链接：** P3406 海底高铁
>
> **难度系数：** ★★

【题目描述】

该铁路经过 N 个城市，每个城市都有一个站。不过，由于各个城市之间不能协调好，于是乘车每经过两个相邻的城市之间（方向不限），必须单独购买这一小段的车票。第 i 段铁路连接了城市 i 和城市 $i+1$。如果搭乘的比较远，需要购买多张车票。第 i 段铁路购买纸质单程票需要 A_i 博艾元。

虽然一些事情没有协调好，各段铁路公司也为了方便乘客，推出了 IC 卡。对于第 i 段铁路，需要花 C_i 博艾元的工本费购买一张 IC 卡，然后乘坐这段铁路一次就只要扣 $B_i(B_i < A_i)$ 元。IC 卡可以提前购买，有钱就可以从网上买得到，而不需要亲自去对应的城市购买。工本费不能退，也不能购买车票。每张卡都可以充值任意数额。对于第 i 段铁路的 IC 卡，无法乘坐别的铁路的车。

Uim 现在需要出差，要去 M 个城市，从城市 p_1 出发分别按照 $P_1, P_2, P_3, ..., P_M$ 的顺序访问各个城市，可能会多次访问一个城市，且相邻访问的城市位置不一定相邻，而且不会是同一个城市。

现在他希望知道，出差结束后，至少会花掉多少的钱，包括购买纸质车票、买卡和充值的总费用。

【输入描述】

第一行两个整数，N, M。

接下来一行，M 个数字，表示 P_i。

接下来 $N-1$ 行，表示第 i 段铁路的 A_i, B_i, C_i。

- 对于 100% 的数据 $M, N \leq 10^5, A_i, B_i, C_i \leq 10^5$

【输出描述】

一个整数，表示最少花费

【示例一】

输入：

9 10

3 1 4 1 5 9 2 6 5 3

200 100 50

300 299 100

500 200 500

345 234 123

100 50 100

600 100 1

450 400 80

2 1 10

输出：

> 6394

【解法】

先考虑如何让花费最小，想要求最小花费，需要知道每一段高铁被「乘坐了多少次」，记作 $f[i]$，那么最小花费就是「买票的花费」与「买卡的花费」两者之间的最小值：

- 买票花费：$a[i] \times f[i]$；
- 买卡花费，乘车花费 + 工本费：$b[i] \times f[i] + c[i]$；
- 那么最小花费就是：$mincost = min(a[i] \times f[i],\ b[i] \times f[i] + c[i])$

接下来考虑如何求出每一段高铁被「乘坐了多少次」。根据访问城市的序列 $p_1, p_2, p_3, ..., p_m$ 可知，对于任意一次访问 $p_i \sim p_{i+1}$，我们会乘坐 $[p_i,\ p_{i+1} - 1]$ 之间所有的高铁，比如 $p_i = 3,\ p_{i+1} = 6$，那么 $[3,\ 5]$ 之间所有的高铁都会被乘坐一次，相当于每个数都加上 1，「注意 6 位置不会乘坐到」。那么我们就可以利用「差分数组」：

- 创建一个全为 0 的差分数组 f；
- 遍历访问序列，对于每一次访问：$f[p_i]++,\ f[p_{i+1}]--$；
- 然后对差分数组做一次前缀和，就得到每个高铁乘坐的次数

注意城市访问的序列有可能 $p_i > p_{i+1}$，此时应该「交换」一下顺序。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

int n, m;
LL f[N]; // 差分数组

int main()
{
    cin >> n >> m;
    // x->y
    int x; cin >> x;
    for(int i = 2; i <= m; i++)
    {
        int y; cin >> y;
```

```
            // x -> y
            if(x > y)
            {
                f[y]++;
                f[x]--;
            }
            else
            {
                f[x]++;
                f[y]--;
            }
            x = y;
        }

        // 利用差分数组，还原出原数组
        for(int i = 1; i <= n; i++) f[i] += f[i - 1];

        // 直接求结果
        LL ret = 0;
        for(int i = 1; i < n; i++)
        {
            LL a, b, c; cin >> a >> b >> c;
            ret += min(a * f[i], c + b * f[i]);
        }

        cout << ret << endl;

        return 0;
    }
```

5.3 二维差分

> **题目来源：** 牛客网
>
> **题目链接：** 【模板】二维差分
>
> **难度系数：** ★

【题目描述】

> 给你一个 n 行 m 列的矩阵，下标从 1 开始。
> 接下来有 q 次操作，每次操作输入 5 个参数 x_1, y_1, x_2, y_2, k，表示把以 (x_1, y_1) 为左上角，(x_2, y_2) 为右下角的子矩阵的每个元素都加上 k。
> 请输出操作后的矩阵。

【输入描述】

第一行包含三个整数 n, m, q。
接下来 n 行，每行 m 个整数，代表矩阵的元素。

接下来 q 行，每行 5 个整数 x_1, y_1, x_2, y_2, k，分别代表这次操作的参数。

$1 \le n, m \le 1000$

$1 \le q \le 10^5$

$1 \le x_1 \le x_2 \le n$

$1 \le y_1 \le y_2 \le m$

$-10^9 \le$ 矩阵中的元素 $\le 10^9$

【输出描述】

输出 n 行，每行 m 个数，每个数用空格分开，表示这个矩阵。

【示例一】

输入：

```
2 3 4
1 2 3
4 5 6
1 1 2 2 3
1 2 2 3 -1
1 1 1 3 4
1 1 2 1 1
```

输出：

```
9 8 6
8 7 5
```

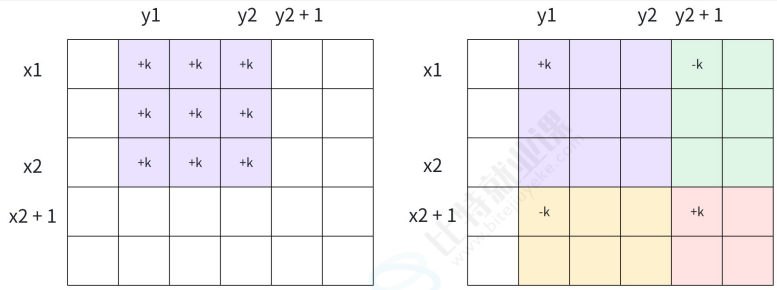
【解法】

二维差分模板题，先根据差分矩阵的「性质」创建差分矩阵，然后根据差分矩阵的「性质」处理 q 次区间修改，最后利用「前缀和」还原出来原始的矩阵。因此，重点就是差分矩阵的「性质」。

可以类比「一维差分数组」的性质，推导出「二维差分矩阵」的性质：

- 在差分数组中某个位置标记：表示后续元素统一被修改；
- 在差分数组中求前缀和：能够还原出原始数组。

假设我们需要将原始矩阵 a 中，以 (x_1, y_1) 为左上角，(x_2, y_2) 为右下角的子矩阵的每个元素都加上 k：



如图所示，在差分矩阵中：
- 将 (x1, y1) 位置加上一个 k，求前缀和之后，会把（紫 + 绿 + 黄 + 粉）区域的所有元素都加上 k；

为了抵消这个 k，我们需要：
- 将 (x1, y2 + 1) 与 (x2 + 1, y1) 位置都减去一个 k，这样（黄 + 绿）区域就恢复正常；

但是，粉色区域在上述操作下会被统一减去一个 k，也需要抵消掉：
- 将 (x2 + 1, y2 + 1) 位置加上一个 k；

这样就可以在差分矩阵求前缀和之后，还原出原始的矩阵。

由此可得差分矩阵的性质：

$$f[x_1][y_1] += k$$

$$f[x_1][y_2 + 1] -= k$$

$$f[x_2 + 1][y_1] -= k$$

$$f[x_2 + 1][y_2 + 1] += k$$

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 1010;

int n, m, q;
LL f[N][N]; // 差分矩阵

```

```cpp
// 差分矩阵的性质
void insert(int x1, int y1, int x2, int y2, LL k)
{
    f[x1][y1] += k; f[x1][y2 + 1] -= k; f[x2 + 1][y1] -= k; f[x2 + 1][y2 + 1] += k;
}

int main()
{
    cin >> n >> m >> q;
    // 预处理差分矩阵
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            LL x; cin >> x;
            // [i, j]为左上角，[i, j]为右下角的矩阵，统一加上 x
            insert(i, j, i, j, x);
        }
    }

    // 处理 q 次修改操作
    while(q--)
    {
        LL x1, y1, x2, y2, k; cin >> x1 >> y1 >> x2 >> y2 >> k;
        insert(x1, y1, x2, y2, k);
    }

    // 利用前缀和还原出修改之后的数组
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= m; j++)
        {
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + f[i][j];
            cout << f[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

5.4 地毯

> **题目来源：** 洛谷
>
> **题目链接：** P3397 地毯
>
> **难度系数：** ★

【题目描述】

> 在 $n \times n$ 的格子上有 m 个地毯。
>
> 给出这些地毯的信息，问每个点被多少个地毯覆盖。

【输入描述】

> 第一行，两个正整数 n, m。意义如题所述。
>
> 接下来 m 行，每行两个坐标 (x_1, y_1) 和 (x_2, y_2)，代表一块地毯，左上角是 (x_1, y_1)，右下角是 (x_2, y_2)。

【输出描述】

> 输出 n 行，每行 n 个正整数。
>
> 第 i 行第 j 列的正整数表示 (i, j) 这个格子被多少个地毯覆盖。

【示例一】

> 输入：
>
> ```
> 5 3
> 2 2 3 3
> 3 3 5 5
> 1 2 1 4
> ```
>
> 输出：
>
> ```
> 0 1 1 1 0
> 0 1 1 0 0
> 0 1 2 1 1
> 0 0 1 1 1
> 0 0 1 1 1
> ```

【解法】

直接利用二维差分矩阵即可。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int f[N][N]; // 差分矩阵

// 差分数组的性质
void insert(int x1, int y1, int x2, int y2, int k)
{
    f[x1][y1] += k; f[x1][y2 + 1] -= k; f[x2 + 1][y1] -= k; f[x2 + 1][y2 + 1] += k;
}

int main()
{
    cin >> n >> m;

    // 构建差分数组
    while(m--)
    {
        int x1, y1, x2, y2; cin >> x1 >> y1 >> x2 >> y2;
        insert(x1, y1, x2, y2, 1);
    }

    // 利用前缀和还原修改之后的数组
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            f[i][j] = f[i - 1][j] + f[i][j - 1] - f[i - 1][j - 1] + f[i][j];
            cout << f[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

6. 双指针

> 双指针算法有时候也叫**尺取法**或者**滑动窗口**，是**一种优化暴力枚举策略的手段**：